

Contents

4. Statički upiti raspona	1
4.1 Pojam	1
4.2 Osnovne tehnike	1
a) Brute force (direktno računanje)	1
b) Prefiksne sume (prefix sums)	2
c) Sparse table (retka tabela)	2
d) Sqrt dekompozicija	3
e) Segmentno drvo	3
4.3 Poređenje tehnika	3
5. Segmentna drveta — pojam, svojstva, formiranje	3
5.1 Pojam	3
5.2 Svojstva	4
5.3 Primer strukture	4
5.4 Formiranje segmentnog drveta — dva pristupa	4
a) Pristup odozgo naniže (top-down, rekurzivan)	4
b) Pristup odozdo naviše (bottom-up, iterativan)	5
5.5 Poređenje pristupa izgradnji	5
6. Segmentna drveta — računanje zbira elemenata iz segmenta	6
6.1 Top-down (rekurzivan) upit zbira	6
Primer	6
6.2 Bottom-up (iterativan) upit zbira	6
Primer (isti niz, isti upit)	7
6.3 Poređenje upita zbira	7
6.4 Kompletan primer (bottom-up, sum + point update)	8
6.5 Zaključak	8

4. Statički upiti raspona

4.1 Pojam

Statički upit raspona (engl. *static range query*) je upit oblika:

“Za dati niz $a[0..n-1]$ koji se **ne menja** (nema update operacija), izračunaj neku agregatnu vrednost (zbir, minimum, maksimum, NZD, XOR...) nad segmentom $[l, r]$.”

Ključna reč je **statički** — niz je fiksiran unapred, pa možemo uložiti vreme u predprocesiranje (preprocessing) da bismo kasnije odgovarali na upite što brže moguće. Ovo je suprotno **dinamičkim upitima raspona**, gde se elementi niza menjaju (update) između upita, pa struktura mora da podržava i ažuriranje (za to se najčešće koristi segmentno drvo ili Fenwick/BIT stablo).

Tipični primeri: - $\text{sum}(l, r) = a_l + a_{l+1} + \dots + a_r$, $\text{min}(l, r)$, $\text{max}(l, r)$, $\text{gcd}(l, r)$, $\text{xor}(l, r)$

4.2 Osnovne tehnike

a) Brute force (direktno računanje)

Za svaki upit prođemo kroz segment $[l, r]$ i akumuliramo rezultat.

```
long long sumQuery(vector<int>& a, int l, int r) {
    long long s = 0;
    for (int i = l; i <= r; i++) s += a[i];
}
```

```
return s;
}
```

- Preprocesiranje: $O(1)$
- Upit: **$O(n)$** u najgorem slučaju
- Za q upita: $O(n \cdot q)$ — neprihvatljivo za velike n, q .

b) Prefiksne sume (prefix sums)

Radi samo za **invertibilne** operacije (sabiranje, XOR) — operacije koje imaju inverz, jer koristimo oduzimanje da “izvadimo” segment iz prefiksa.

Definišemo $P[i] = a_0 + a_1 + \dots + a_{i-1}$ (prefiks dužine i), $P[0] = 0$.

$$\text{sum}(l, r) = P[r + 1] - P[l]$$

```
vector<long long> P(n + 1, 0);
for (int i = 0; i < n; i++) P[i + 1] = P[i] + a[i];

long long sumQuery(int l, int r) { return P[r + 1] - P[l]; }
```

- Preprocesiranje: **$O(n)$**
- Upit: **$O(1)$**
- Nedostatak: ne radi za min/max/gcd (te operacije nemaju inverz — ne možemo “oduzeti” minimum).

c) Sparse table (retka tabela)

Radi za **idempotentne** operacije (operacija $\circ$ je idempotentna ako $x \circ x = x$) — min, max, gcd, AND, OR. Kod ovih operacija upit se sme “preklapati” jer ponavljanje elementa ne kviri rezultat.

Ideja: unapred izračunamo $\text{sp}[k][i]$ = agregat nad segmentom dužine 2^k koji počinje na poziciji i . Upit (l, r) delimo na **dva preklapajuća** segmenta dužine 2^k , gde je $k = \lfloor \log_2(r - l + 1) \rfloor$:

$$\min(l, r) = \min(\text{sp}[k][l], \text{sp}[k][r - 2^k + 1])$$

```
int K = log2(n) + 1;
vector<vector<int>>> sp(K, vector<int>(n));
sp[0] = a; // baza: segmenti dužine 1

for (int k = 1; k < K; k++)
    for (int i = 0; i + (1 << k) <= n; i++)
        sp[k][i] = min(sp[k-1][i], sp[k-1][i + (1 << (k-1))]);

int minQuery(int l, int r) {
    int k = log2(r - l + 1);
    return min(sp[k][l], sp[k][r - (1 << k) + 1]);
}
```

- Preprocesiranje: **$O(n \log n)$**
- Upit: **$O(1)$**
- Radi samo za idempotentne operacije i samo kada nema ažuriranja.

d) Sqrt dekompozicija

Niz delimo na blokove veličine $\approx \sqrt{n}$ i za svaki blok čuvamo agregat. Upit obrađuje cele blokove u $O(1)$ po bloku i granične elemente pojedinačno.

- Preprocesiranje: $O(n)$
- Upit: **$O(\sqrt{n})$**
- Fleksibilnija je od sparse table (lako podržava i ažuriranje), ali sporija od segmentnog drveta.

e) Segmentno drvo

Univerzalna struktura koja radi za **bilo koju asocijativnu operaciju** (sum, min, max, gcd, xor...), sa ili bez ažuriranja.

- Preprocesiranje (izgradnja): $O(n)$
- Upit: **$O(\log n)$**
- Ažuriranje (ako je potrebno): **$O(\log n)$**

Ovoj strukturi je posvećen ostatak teksta.

4.3 Poređenje tehnika

Tehnika	Preprocesiranje	Upit	Podržava update	Ograničenje na operaciju
Brute force	$O(1)$	$O(n)$	da, $O(1)$	bilo koja
Prefiksne sume	$O(n)$	$O(1)$	ne ($O(n)$ za update)	invertibilna (sum, xor)
Sparse table	$O(n \log n)$	$O(1)$	ne	idempotentna (min, max, gcd)
Sqrt dekompozicija	$O(n)$	$O(\sqrt{n})$	da, $O(\sqrt{n})$ ili $O(1)$	asocijativna
Segmentno drvo	$O(n)$	$O(\log n)$	da, $O(\log n)$	asocijativna

Zaključak: ako je niz zaista statičan i operacija je sum $\rightarrow$ koristiti prefiksne sume (najbrže i najjednostavnije); ako je operacija idempotentna (min/max/gcd) $\rightarrow$ sparse table; ako će biti ažuriranja ili operacija nije ni invertibilna ni idempotentna $\rightarrow$ segmentno drvo.

5. Segmentna drveta — pojam, svojstva, formiranje

5.1 Pojam

Segmentno drvo (engl. *segment tree*) je binarno stablo u kome:

- svaki **list** predstavlja jedan element originalnog niza $a[0..n-1]$,
- svaki **unutrašnji čvor** predstavlja **segment (interval)** niza jednak uniji segmenata svoje dve dece, i čuva agregatnu vrednost te operacije (npr. zbir, minimum) nad tim segmentom,
- **koren** predstavlja ceo niz $[0, n-1]$.

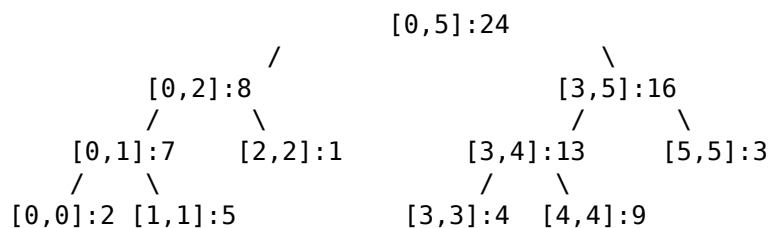
Segment se u svakom koraku deli na (približno) dve polovine — levo i desno dete pokrivaju $[l, mid]$ i $[mid+1, r]$, gde je $mid = \lfloor (l+r)/2 \rfloor$.

5.2 Svojstva

1. **Visina stabla** je $O(\log n)$, jer se segment prepolovljava u svakom nivou.
2. **Broj čvorova** je $O(n)$ — tačnije, stablo ima najviše $2n - 1$ čvorova (kao svako puno binarno stablo sa n listova), ali se u praksi zbog jednostavnosti implementacije alokira niz veličine $4n$ (kod rekurzivne/top-down varijante) da bi se izbeglo računanje tačne veličine za proizvoljno n .
3. **Asocijativnost operacije** je jedini zahtev — segmentno drvo radi za sum, min, max, gcd, xor, proizvod po modulu itd., jer se svaki čvor računa kao kombinacija dece: $\text{node} = \text{node}_{\text{levo}} \circ \text{node}_{\text{desno}}$.
4. Podržava i **upite** (query) i **ažuriranja** (update) tačke ili opsega u $O(\log n)$ (opseg-update zahteva dodatnu tehniku *lazy propagation*, van okvira ovog teksta).
5. Segmentno drvo se može predstaviti **implicitno kao niz** (bez pokazivača), koristeći indeksiranje slično binarnoj gomili (heap): za čvor na indeksu i , deca su na indeksima $2i$ i $2i + 1$ (1-indeksirano) ili $2i + 1, 2i + 2$ (0-indeksirano).

5.3 Primer strukture

Za niz $a = [2, 5, 1, 4, 9, 3]$ ($n = 6$), stablo (top-down, sum-agregat) izgleda ovako (prikazano kao $[l, r] : \text{vrednost}$):



Svaki unutrašnji čvor je zbir svoje dece: npr. $[0, 2] = [0, 1] + [2, 2] = 7 + 1 = 8$.

5.4 Formiranje segmentnog drveta — dva pristupa

a) Pristup odozgo naniže (top-down, rekurzivan)

Ovo je klasičan, najčešće korišćen pristup. Stablo se gradi **rekurzivno**: počinjemo od korena koji predstavlja ceo opseg $[0, n - 1]$, delimo ga na dva podopsega, rekurzivno gradimo levo i desno podstablo, a zatim kombinujemo njihove vrednosti.

```
vector<long long> tree; // veličine 4*n
vector<int> a;

void build(int node, int l, int r) {
    if (l == r) {
        tree[node] = a[l]; // list
        return;
    }
    int mid = (l + r) / 2;
    build(2*node, l, mid); // levo dete
    build(2*node+1, mid+1, r); // desno dete
    tree[node] = tree[2*node] + tree[2*node+1]; // kombinacija
}
```

- Poziva se kao `build(1, 0, n-1)` (koren je indeks 1).
- Rekurzija ide **od korena ka listovima** (deli pa vladaj — silazi u dubinu), a vrednosti se popunjavaju **pri povratku iz rekurzije** (postorder), odnosno “odozdo naviše” u smislu redosleda računanja, ali je struktura izgradnje top-down.

- Složenost: **$O(n)$** (svaki čvor se poseti tačno jednom).
- Memorija: $4n$ (bezbedna gornja granica za rekurzivnu implementaciju, jer stablo ne mora biti savršeno balansirano kada n nije stepen dvojke).

b) Pristup odozdo naviše (bottom-up, iterativan)

Popularizovao ga je AtCoder Library / poznat kao “iterativni segment tree”. Niz veličine $2n$: pozicije $[n, 2n - 1]$ su **listovi** (originalni elementi), a pozicije $[1, n - 1]$ su unutrašnji čvorovi. Roditelj čvora i je $i/2$, deca su $2i$ i $2i + 1$.

```
int n;
vector<long long> tree; // veličine 2*n

void build(vector<int>& a) {
    n = a.size();
    tree.assign(2 * n, 0);
    // 1. korak: upiši listove
    for (int i = 0; i < n; i++)
        tree[n + i] = a[i];
    // 2. korak: idi od poslednjeg unutrašnjeg čvora ka korenu
    for (int i = n - 1; i >= 1; i--)
        tree[i] = tree[2*i] + tree[2*i + 1];
}
```

- Nema rekurzije — **direktno se popunjava niz**, prvo listovi ($n..2n-1$), zatim se **iterativno** ide unazad ka korenu ($n-1$ do 1), pa otuda naziv “odozdo naviše”.
- Složenost: **$O(n)$** .
- Memorija: tačno $2n$ — efikasnije od top-down varijante.
- Ograničenje: najjednostavnije radi kada je n proizvoljno (ne mora biti stepen dvojke), ali segmenti koje pojedini unutrašnji čvorovi predstavljaju nisu uvek “lepi” kontinualni opsezi jednake dužine kada n nije stepen dvojke (ipak, upiti i dalje rade ispravno zahvaljujući načinu obilaska objašnjenom u sekciji 6).

5.5 Poređenje pristupa izgradnji

	Top-down (rekurzivno)	Bottom-up (iterativno)
Implementacija	Rekurzija, intuitivnija za razumevanje	Iterativna, kompaktnija
Memorija	$\sim 4n$	tačno $2n$
Brzina konstante	nešto sporije (rekurzija, poziv funkcije)	brže u praksi (bez rekurzije, keš-prijateljski niz)
Lako proširiti na lazy propagation (opseg-update)	da, prirodno	teže, potrebne dodatne tehnike
Podrška proizvoljnog n	da	da, ali sa manje “urednom” strukturom segmenata

6. Segmentna drveta — računanje zbira elemenata iz segmenta

6.1 Top-down (rekurzivan) upit zbira

Da izračunamo $\text{sum}(l, r)$, rekurzivno obilazimo stablo počevši od korena $\text{query}(1, 0, n-1, l, r)$. U svakom čvoru koji predstavlja segment $[tl, tr]$ razlikujemo tri slučaja:

1. **Nema preklapanja:** $[tl, tr] \cap [l, r] = \emptyset \rightarrow$ vrati neutralni element (0 za sumu).
2. **Potpuno preklapanje:** $[l, r] \subseteq [tl, tr] \rightarrow$ vrati $\text{tree}[\text{node}]$ direktno (ne ulazimo dublje).
3. **Delimično preklapanje:** rekurzivno upitaj oba deteta i saberi rezultate.

```
long long query(int node, int tl, int tr, int l, int r) {
    if (r < tl || tr < l) return 0;           // 1) nema preklapanja
    if (l <= tl && tr <= r) return tree[node]; // 2) potpuno preklapanje
    int mid = (tl + tr) / 2;                 // 3) delimično preklapanje
    return query(2*node, tl, mid, l, r)
           + query(2*node+1, mid+1, tr, l, r);
}
```

Poziva se: $\text{query}(1, 0, n-1, l, r)$.

Zašto je ovo $O(\log n)$? Može se pokazati da postoji najviše $O(\log n)$ čvorova na svakom nivou koji se "delimično preklapaju" i moraju dalje da se granaju — svi ostali upiti se ili odbacuju u $O(1)$ ili vraćaju kompletnu vrednost čvora u $O(1)$. Ukupan broj posećenih čvorova je $O(\log n)$.

Primer

Za $a = [2, 5, 1, 4, 9, 3]$ (stablo iz sekcije 5.3), izračunajmo $\text{sum}(1, 4)$:

- $\text{query}([0, 5])$: delimično preklapanje $\rightarrow$ grana se na $[0, 2]$ i $[3, 5]$
- $\text{query}([0, 2])$: delimično $\rightarrow$ grana se na $[0, 1]$ i $[2, 2]$
 - $\text{query}([0, 1])$: delimično $\rightarrow$ grana se na $[0, 0]$ i $[1, 1]$
 - * $[0, 0] = \{0\}$: van $[1, 4] \rightarrow 0$
 - * $[1, 1] = \{1\}$: unutar $[1, 4] \rightarrow 5$
 - $[2, 2] = \{2\}$: unutar $[1, 4] \rightarrow 1$
 - ukupno za $[0, 2]$: $0 + 5 + 1 = 6$
- $\text{query}([3, 5])$: delimično $\rightarrow$ grana se na $[3, 4]$ i $[5, 5]$
 - $[3, 4]$: potpuno unutar $[1, 4] \rightarrow$ vraća tree vrednost direktno = 13
 - $[5, 5] = \{5\}$: van $[1, 4] \rightarrow 0$
 - ukupno za $[3, 5]$: $13 + 0 = 13$
- **Ukupan rezultat:** $6 + 13 = 19$

Provera: $a_1 + a_2 + a_3 + a_4 = 5 + 1 + 4 + 9 = 19$. (potvrđeno) Podudara se, a čvor $[3, 4]$ je iskorišćen direktno bez ulaska u listove — to je suština efikasnosti.

6.2 Bottom-up (iterativan) upit zbira

Kod iterativne (bottom-up) reprezentacije, listovi su na pozicijama $[n, 2n - 1]$. Upit $\text{sum}(l, r)$ (0-indeksirano, uključivo) implementira se pomeranjem dva pokazivača od listova ka korenu:

```
long long query(int l, int r) { // [l, r] uključivo
    long long res = 0;
    for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
        if (l & 1) res += tree[l++]; // l je desno dete -> uzmi ga, pomeri se
        if (r & 1) res += tree[--r]; // r je desno dete -> pomeri se, uzmi ga
    }
}
```

```

return res;
}

```

Objašnjenje ideje: - Pomerimo l i r na indekse listova: $l += n$, a r postaje “isključiva” granica $r += n + 1$. - U svakoj iteraciji, ako je l **desno dete** svog roditelja (neparan indeks), taj čvor **ne može** biti obuhvaćen zajedno sa svojim bratom u sledećem nivou (jer bi to prevazišlo levu granicu), pa ga direktno dodajemo u rezultat i pomeramo l u desno. - Analogno, ako je r (ekskluzivno) **desno dete**, znači da čvor **levo od njega** ($r-1$) treba uključiti direktno. - Zatim se l i r “penju” na roditeljski nivo ($l \neq 2$, $r \neq 2$) i postupak se ponavlja dok se pokazivači ne sretnu.

Ova tehnika automatski bira **minimalan skup** čvorova koji tačno pokriva $[l, r]$, analogno onome što rekurzivna verzija radi kroz grananje, ali bez rekurzije — čisto iterativno, penjanjem uz stablo.

Primer (isti niz, isti upit)

$a = [2, 5, 1, 4, 9, 3]$, $n = 6$, tražimo $\text{sum}(1, 4)$.

Niz tree (bottom-up, 1-indeksirano, pozicije 6..11 su listovi):

```

tree[6..11] = [2, 5, 1, 4, 9, 3]
tree[3] = tree[6]+tree[7] = 7      (segment [0,1])
tree[4] = tree[8]+tree[9] = 5      (segment [2,3])
tree[5] = tree[10]+tree[11] = 12   (segment [4,5])
tree[2] = tree[4]+tree[5] = 17     (segment [2,5])
tree[1] = tree[2]+tree[3] = 24     (segment [0,5])

```

Inicijalno: $l = 1 + 6 = 7$, $r = 4 + 6 + 1 = 11$.

Iteracija	l	r	l neparno?	r neparno?	Akcija	res
1	7	11	da (7)	da (11)	$\text{res} +=$ $\text{tree}[7]=5$, $l=8$; $\text{res} +=$ $\text{tree}[10]=9$, $r=10$	14
—	$l=8/2=4$	$r=10/2=5$				
2	4	5	ne	da (5)	$\text{res} +=$ $\text{tree}[4]=5$, $r=4$	19
—	$l=4/2=2$	$r=4/2=2$			petlja staje ($l < r$ ne važi, $2 < 2$ false)	

Rezultat: 19 — isti kao kod top-down pristupa. (potvrđeno)

6.3 Poređenje upita zbira

	Top-down (rekurzivno)	Bottom-up (iterativno)
Vremenska složenost	$O(\log n)$	$O(\log n)$
Rekurzija	da	ne

	Top-down (rekurzivno)	Bottom-up (iterativno)
Praktična brzina	nešto sporije (poziv funkcije, uslovno grananje)	brže — čist iterativan kod, dobro se keš-uje
Čitljivost / lakoća proširenja (lazy propagation, range update)	lakše	teže, potrebne dodatne modifikacije
Tipično se koristi u	edukativnim materijalima, kompleksnijim segment tree varijantama	competitive programming (brzina), kada nema potrebe za range-update

6.4 Kompletan primer (bottom-up, sum + point update)

```
class SegmentTreeSum {
    int n;
    vector<long long> tree;
public:
    SegmentTreeSum(vector<int>& a) {
        n = a.size();
        tree.assign(2 * n, 0);
        for (int i = 0; i < n; i++) tree[n + i] = a[i];
        for (int i = n - 1; i >= 1; i--)
            tree[i] = tree[2*i] + tree[2*i+1];
    }

    void update(int pos, int value) {           // O(log n)
        pos += n;
        tree[pos] = value;
        for (pos /= 2; pos >= 1; pos /= 2)
            tree[pos] = tree[2*pos] + tree[2*pos+1];
    }

    long long query(int l, int r) {             // O(log n), [l, r] uključivo
        long long res = 0;
        for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
            if (l & 1) res += tree[l++];
            if (r & 1) res += tree[--r];
        }
        return res;
    }
};
```

Upotreba:

```
vector<int> a = {2, 5, 1, 4, 9, 3};
SegmentTreeSum st(a);

cout << st.query(1, 4) << "\n"; // 19
st.update(2, 10);                // a[2] postaje 10 (bilo 1)
cout << st.query(1, 4) << "\n"; // 5 + 10 + 4 + 9 = 28
```

6.5 Zaključak

- Za **statičke** upite sume, prefiksne sume su i dalje najjednostavnije i najbrže rešenje ($O(1)$ po upitu).

- Segmentno drvo postaje neophodno kada operacija nije invertibilna/idempotentna, ili kada su potrebna **ažuriranja** niza između upita.
- Top-down (rekurzivan) pristup je intuitivniji i lakše se proširuje (npr. na lazy propagation za range update), dok je bottom-up (iterativan) pristup kompaktniji i brži u praksi za osnovne operacije (point update, range query).
- Oba pristupa garantuju **$O(\log n)$** po upitu i **$O(n)$** za izgradnju, što segmentno drvo čini jednom od najkorisnijih struktura podataka u kompetitivnom programiranju.